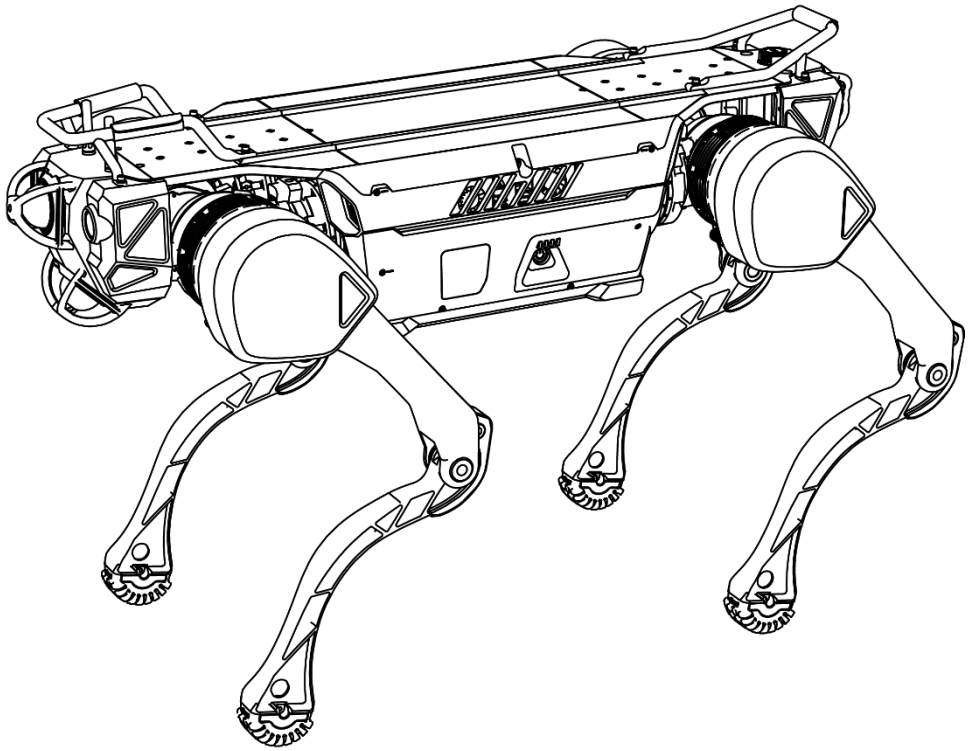
DEEPRobotics

Jueying X30

API Documentation (beta)

V1.0.5-0 August 21, 2025



CATALOG

1 UDP Command 6

1.1 Protocol Format 6

1.1.1 Simple Command 6

1.1.2 Complex Command 7

1.1.3 Complex Command Reception 8

1.2 Send Command to Motion Host 8

1.2.1 Heartbeat 9

1.2.2 Basic State Conversion Commands 9

1.2.3 Axis Command 10

1.2.4 Control Mode Command 14

1.2.5 Body Height Switching Command 14

1.2.6 Gait Command 15

1.2.7 Save Data Command 16

1.2.8 Soft Emergency Stop 16

1.3 Receive Message from Motion Host 16

1.3.1 Robot Running Status 16

1.3.2 Robot Motion Status 20

1.3.3 Motion Control Sensor Data 23

1.3.4 Motion Control System Status 25

1.3.5 Battery Information 25

1.3.6 Body Height Status 26

1.4 Command Set of Autonomous Charging 27

1.5 Command Set of Elevation Map Module 29

1.5.1 Velocity Input Source 29

1.5.2 Elevation Map Mode 31

1.5.3 Obstacle Avoidance Modes 36

1.5.4 Obstacle Height Threshold 36

2 Perception Host ROS Topics 38

2.1 Topics of Sensor Driver 38

2.2 Topics of Basic Status Information 39

2.2.1 Battery Information 39

2.2.2 Motion State 40

2.2.3 Joint State 43

2.2.4 Running State 43

2.3 Velocity-Related Topics 44

2.3.1 Velocity Commands from Joystick 44

2.3.2 Velocity Commands from Navigation Module 44

2.3.3 Velocity Commands Corrected by Elevation Map Module 45

2.4 Topics of Elevation Map Module 45

2.4.1 Access to Elevation Map Data 45

2.4.2 Obstacle Avoidance Mode 46

2.4.3 Velocity Input Source 46

2.4.4 Elevation Map Mode 48

2.4.5 Obstacle Height Threshold 53

2.4.6 Velocity Correction Time Threshold 54

2.4.7 Braking Time Threshold 54

2.5 Topics of Autonomous Charging 55

3 Navigation Host ROS Topics 57

3.1 Topics of Sensor Driver 57

3.2 Topics of Basic Status Information 57

3.3 Topics of Localization 57

3.3.1 Reset Robot Pose 57

3.3.2 Change Global Map 57

3.3.3 Obtain Real-time Pose 57

3.3.4 Get Global Point Cloud Map Data 58

3.3.5 Obtain Location Status 58

3.4 Topics of Navigation 59

3.4.1 Get Occupancy Grid Map Data 59

3.4.2 Specify Auto Navigation Goals 59

3.4.3 Specify Straight Navigation Goals 60

3.4.4 Get Global Path Planned by Auto Navigation 61

3.4.5 Get Local Path Planned by Auto Navigation 61

3.4.6 Get Global Path Intercepted By Auto Navigation Local Planner 62

3.4.7 Get Obstacle Avoidance Status of Navigation 62

3.4.8 Obtain or Publish Navigation Velocity Commands 62

Appendix A: UDP Message Parsing 64

Appendix B: Maximum Velocity of Each Gait 65

Appendix C: How to Use Multi-Frame Mode via UDP 66

1 UDP Command

The communication between the motion host and the perception host (or controller) adopts UDP protocol, which stores the raw data in little endian.

1.1 Protocol Format

This section contains the introductions of the raw data format of UDP transmission. Based on whether carrying the complex data, UDP commands can be divided into two categories: Simple command and complex command.

1.1.1 Simple Command

The format of simple command is `[xxxx yyyy zzzz]`, and one letter represents one byte. Simple commands use the structure `CommandHead` to store the content of raw data.

```
struct CommandHead{
    uint32_t code;
    uint32_t paramters_size;
    uint32_t type;
};
```

xxxx is the value of `CommandHead.code`, which represents the command code.

yyyy is the value of the `CommandHead.paramters_size`, representing the command value of the command code. If the command code has no valid command value, yyyy = 0.

zzzz is the value of `CommandHead.type`, which is used to distinguish the command types. The type value of simple command is 0, that is, zzzz = 0.

The use case is as follows:

```
//routine
struct CommandHead command_head = {0};
command_head.code = 1;            //Command code
```

```
command_head.paramters_size = 0;// Command value
command_head.type = 0;           // Command type
sendto(sfd,&command_head,sizeof(command_head),Target address, address size )
```

1.1.2 Complex Command

Complex commands carry the specific data content in the format of `[xxxx yyyy zzzz bbbbbbbb......]`, where one letter represents one byte.

Complex commands use the structure `Command` to store the content of raw data.

```
struct CommandHead{
    uint32_t code;
    uint32_t paramters_size;
    uint32_t type;
};
const uint32_t kDataSize = 256;
struct Command{
    CommandHead head;
    uint32_t data[kDataSize];
};
```

xxxx is the value of `Command.head.code`, which represents the command code.

yyyy is the value of the `Command.head.paramters_size`, representing the length of the data content bbbbbbbb.....

zzzz is the value of `Command.head.type`, which is used to distinguish the command types. The type value of complex command is 1, that is, zzzz = 1.

bbbbbbbb..... is the value of `Command.data`, which represents the data content carried by this command.

The use case is as follows:

```
//routine
int32_t to_be_send_data[64];
struct Command command = {0};
command.head.code = 52;   // Command code
command.head.paramters_size = sizeof(to_be_send_data);// Data length
command.head.type = 1;         // Command type
```

```
memcpy(&command.data,&to_be_send_data,sizeof(to_be_send_data));
sendto(sfd,&command,sizeof(command.head)+command.head.paramters_size,Target address, address size);
```

1.1.3 Complex Command Reception

Taking joint velocity as an example:

```
struct RobotJointVel{
    double joint_vel[12];
}; // Taking joint velocity as an example
struct CommandMessage{
    CommandHead command;
    uint8_t data_buffer[1024];
};
CommandMessage cm;
size_t recv_size = 0;
recv_size = recvfrom(server_fd,&cm,sizeof(cm),0,(sockaddr*)&client_addr,&sockaddr_in_size);
if(cm.command.type == 1){ // Judge whether it is the complex command
    if(recv_size == cm.command.paramters_size + sizeof(cm.command)){
        uint8_t* temp_values = new uint8_t[cm.command.paramters_size];
        memcpy(temp_values,cm.data_buffer,cm.command.paramters_size);
        if(cm.command.code == 0x0903){ // Judge the command code and parse the data
            RobotJointVel *joint_vel = (RobotJointVel *)temp_values;
        }
    }
}
```

1.2 Send Command to Motion Host

The controller and perception host directly issue commands to the motion host through UDP communication, so as to control the robot to achieve corresponding functions.

The target IP and port for the perception host or controller to send messages to the motion host are: `192.168.1.103:43893`.

However, please note that sending a message does not change the original control logic. For example, when the robot is lying down, the command of start motion does not work, and the control logic still follows the robot's own motion logic.

[Notes] All command codes in hexadecimal format introduced in this section use the first two digits to distinguish the sender source:
When sending command codes from a remote control terminal (e.g., a controller), the first two digits should be 0x21 (e.g., 0x21040001).
When command codes are issued by autonomous movement algorithm modules (e.g., a navigation module), the first two digits should be 0x31 (e.g., 0x31040001).
In subsequent descriptions, command codes will be presented with 0x21 as the default starting prefix unless otherwise specified.

1.2.1 Heartbeat

The heartbeat is used to confirm whether the connection is normal. Please follow the steps below to send heartbeat.

1. To maintain the connection, continuously send the following heartbeat command code at a frequency of no less than 2 Hz. If a transmission timeout occurs, the motion program will assume the connection is disconnected:

Command code	Command value	Command type
0x21040001	-	0

[Notes] The minus sign "-" in the command value indicates that this value does not work, that is, the command value will not have any effect on the command.

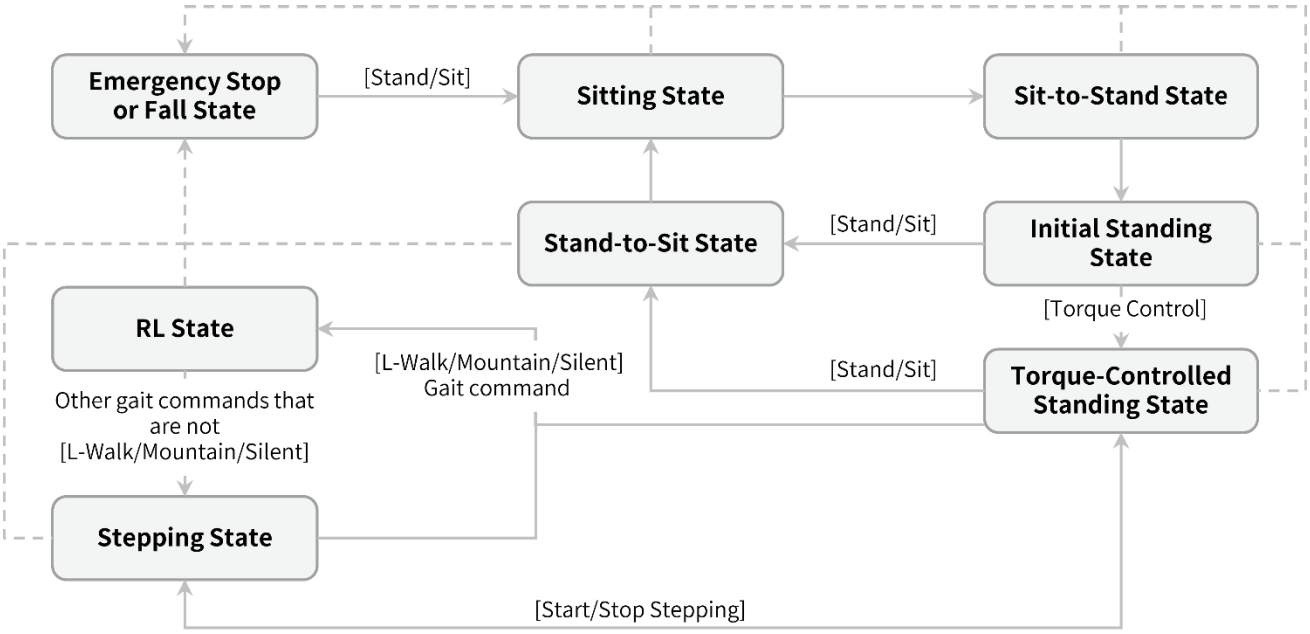
2. After initiating continuous transmission of the heartbeat command code 0x21040001, you must send the following instruction once to confirm the connection:

Command code	Command value	Command type
0x21020001	-	0

1.2.2 Basic State Conversion Commands

The basic state of the robot is converted by corresponding commands. The figure below

shows the conversion of the basic state of the robot. The dotted arrow in the figure indicates that in any basic state, the robot may convert from its current state to the Soft Emergency Stop/Fall Down state due to an unexpected accident.



The commands involved in the state conversion in the figure are shown below:

Command	Code	Value	Type	Function
Stand/Sit	0x21010202	-	0	Alternate between Sitting and Standing
Torque-control	0x2101020A	-	0	Switch from Initial Standing state to Torque-controlled Standing state
Start/Stop stepping	0x21010201	-	0	Alternate between Torque-controlled Standing state and Stepping state

[Notes] The robot's gait commands can be referenced in 1.2.6.

1.2.3 Axis Command

Axis command is the command output from x-axis and y-axis of the two joysticks of the controller, with a value range of: $[-32767,32767]$. The issue frequency of axis command should be 50Hz. If the motion host has not received any axis command for more than one second, the command will be considered as invalid by the system.

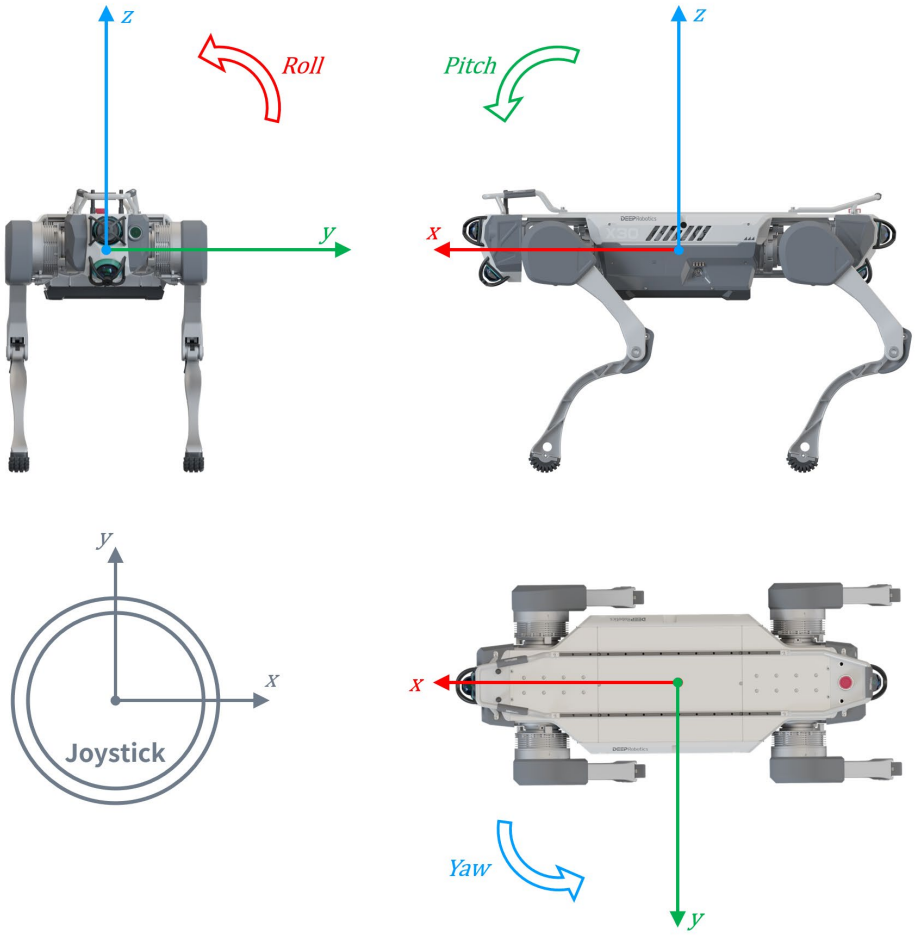
1.2.3.1 Axis Command in Troque-controlled Standing State

When the robot is in the Torque-controlled Standing state, the axis command can issued to change the robot pose. If the command times out, the robot will resume its normal standing pose.

Command	Code	Type	Function
Left joystick X-axis	0x21010131	0	Change Roll angle: Raise the left shoulder in case of the positive value, and raise the right shoulder in case of the negative value
Left joystick Y-axis	0x21010130	0	Change body height: Raise the body in case of the positive value, and lower the body in case of the negative value
Right joystick X-axis	0x21010135	0	Change Yaw angle: Turn the head to the right in case of the positive value, and to the left in case of the negative value
Right joystick Y-axis	0x21010102	0	Change Pitch angle: Head up in case of the positive value, and head down in case of the negative value

1.2.3.2 Axis Command in Stepping State

When the robot is in the Stepping state, its velocity and direction can be adjusted through axis command. Robot can accept the axis command issued by users and map it from the joystick coordinate to body coordinate. Joystick coordinate and body coordinate are shown as follow (the curved arrows indicate the positive direction of rotation).



If the command times out, the robot will decelerate to zero.

1) Y-axis of left joystick:

Code	Type	Function
0x21010130	0	Specify the expected linear velocity on the X-axis of body coordinate. The positive command value for forward translation and the negative for backward translation.

The relationship between the expected linear velocity in the X-axis of body coordinate system and the command value of the left joystick Y-axis is as follows.

$$v_x = \begin{cases} 0, & -655 < y < 655 \\ \frac{y}{32767} \times v_{max}, & 655 \leq |y| \leq 32767 \end{cases}$$

v_x represents the expected linear velocity in the X-axis of body coordinate system, v_{max} represents the maximum speed of the current gait (The maximum speed of each gait can be found in Appendix B), and y represents the command value of the left joystick Y-axis.

2) X-axis of left joystick:

Code	Type	Function
0x21010131	0	Specify the expected linear velocity on the Y-axis of body coordinate. The positive command value for right translation and the negative for left translation.

The relationship between the expected linear velocity in the Y-axis of body coordinate system and the command value of the left joystick X-axis is as follows.

$$v_y = \begin{cases} 0, & -655 < x < 655 \\ (-1) \times \frac{x}{32767} \times v_{max}, & 655 \leq |x| \leq 32767 \end{cases}$$

v_y represents the expected linear velocity in the Y-axis of body coordinate system, v_{max} represents the maximum speed of the current gait, and x represents the command value of the left joystick X-axis.

3) X-axis of right joystick:

Code	Type	Function
0x21010135	0	Specify the expected Yaw angular velocity. The positive command value for turning right and the negative for turning left.

The relationship between the expected Yaw angular velocity and the command value of the right joystick X-axis is as follows.

$$\omega = \begin{cases} 0, & -655 < x < 655 \\ (-1) \times \frac{x}{32767} \times \omega_{max}, & 655 \leq |x| \leq 32767 \end{cases}$$

ω represents the expected Yaw angular velocity, ω_{max} represents the maximum steering speed of the current gait (The maximum steering speed of each gait can be found in Appendix B), and x represents the command value of the right joystick X-axis.

1.2.4 Control Mode Command

The control mode determines the source of the velocity command that the robot responds to. In the non-manual mode, the robot responds to velocity commands from the perception host; and in the manual mode, the robot responds to velocity commands from the controller.

Command	Code	Value	Type	Function
Non-Manual mode	0x21010C03	-	0	Switch the robot from the manual mode to the non-manual mode
Manual mode	0x21010C02	-	0	Switch the robot from the non-manual mode to the manual mode

1.2.5 Body Height Switching Command

The robot supports switching body height.

Command	Code	Value	Type	Function
Height Switching	0x21010406	0=Crawl, 2=Normal	0	Switching body height gears

[Note] When the robot is in the off-road gait, stair gait, or run gait, do not switch the body height to the crawl height.

[Note] Please refer to 1.3.6 to receive the message of the body height status.

1.2.6 Gait Command

When the robot is in stepping state, users can issue gait command to change its gait according to different terrain.

Command	Code	Value	Type
Walk	0x21010300	-	0
Slope	0x21010402	-	0
Off-Road	0x21010401	-	0
Stair (elevation map mode = solid ground/grid ground/stair without rises)	0x21010405	-	0
Stair (elevation map mode= multi-frame mode)	0x2101040A	-	0
45°Stair (elevation map mode= multi-frame mode)	0x2101040B	-	0
L-Walk	0x21010420	-	0
Mountain	0x21010421	-	0
Silent	0x21010422	-	0

[Note] The three types of stair gait can only be used in the corresponding elevation map mode. Please refer to 1.5.2 to change the elevation map mode.

[Note] When the robot is in crawl height, only walk gait and slope gait are supported, please do not switch to other gaits in crawl height.

[Note] Switching the robot's body height and terrain is not supported when the robot is using the L-Walk, Mountain, and Silent gait.

1.2.7 Save Data Command

By the Save Data function, the robot can, in the event of malfunction, save data within the last 5 minutes and automatically exit the motion program.

If the *jy_exe* program is version v2.2.45 or higher:

Command code	Command value	Command type
0x010C01	-	0

If the *jy_exe* program is below version v2.2.45:

Command code	Command value	Command type
18	-	0

1.2.8 Soft Emergency Stop

Soft emergency stop can make the robot immediately lie down and enter the joint protection state.

Command code	Command value	Command type
0x21010C0E	-	0

1.3 Receive Message from Motion Host

The motion host will report status information and joint information to the network addresses listed in *network.toml*. Users can refer to the "4.3.2 Network Parameters" section in *Application Manual* to add required addresses. Then the following information can be received.

1.3.1 Robot Running Status

Command Code	Command Type	Data content	Frequency
0x1008	1	Structure `RcsData` (see below)	200 Hz

```
struct RcsData{
  char robot_name[15];
  int32_t current_mileage;//unit :cm
  int32_t total_mileage;//unit :cm
  long current_run_time;//unit :s
  long total_run_time;//unit :s
  long current_motion_time;//unit :s
  long total_motion_time;//unit :s
  float joystick[4];
  union {
    struct {
      uint8_t is_nav_mode;
      uint8_t emergency_source;
      uint8_t mode_reserved[8];
    }rcs_state_list;
    uint8_t rcs_state[10];
  };
  union{
    struct{
      uint32_t imu_error : 1;
      uint32_t wifi_error : 1;
      uint32_t driver_heat_warn : 1;
      uint32_t driver_error: 1;
      uint32_t motor_heat_warn : 1;
      uint32_t battery_low_warn : 1;
      uint32_t battery_heat_warn : 1;
      uint32_t gpio_error : 1;
      uint32_t cpu_heat_warn : 1;
      uint32_t cpu_freq_warn : 1;
      uint32_t reserved : 22;
    }error_state_bit;
    uint32_t error_state;
  };
};
```

Fields	Type	Meaning
robot_name	char[15]	Robot name

Fields	Type	Meaning
current_mileage	int32_t	Mileage since last boot (cm)
total_mileage	int32_t	Total mileage (cm)
current_run_time	long	Running time since last boot (s)
total_run_time	long	Total running time (s)
current_motion_time	long	Motion time since last boot (s)
total_motion_time	long	Total motion time (s)
joystick	float[4]	Axis command value scaled to [1.0, 1.0][1]
rcs_state	uint8_t[10]	Status[2]
error_state	uint32_t	Error code[3]

[1]The parameters of `joystick` are defined as follows:

Parameter	Type	Range	Meaning
LX	float	[-1.0,1.0]	Left joystick X axis command value
LY	float	[-1.0,1.0]	Left joystick Y axis command value
RX	float	[-1.0,1.0]	Right joystick X axis command value
RY	float	[-1.0,1.0]	Right joystick Y axis command value

[Note]The axis command value here is scaled from the original value range $[-32767,32767]$ to $[-1.0,1.0]$.

[2]The parameters of `rcs_state` are defined as follows:

Parameter	Type	Value
is_nav_mode	uint8_t	0=Manual mode, 1=Non-manual mode

Parameter	Type	Value
emergency_source	uint8_t	1 = Tail soft emergency stop triggered 2 = Soft emergency stop autonomously triggered during motion 3 = Soft emergency stop autonomously triggered during navigation 4 = Soft emergency stop autonomously triggered during autonomy-ous charging 5 = Soft emergency stop triggered by client 6 = Soft emergency stop autonomously triggered by RAS
mode_reserved	uint8_t[9]	Reserve

[3]Each bit of `error_state` corresponds to a kind of error. From the lowest to the highest bit, each bit stands for:

Error Type	Data Type	Value of the bit
imu_error	bool	0=Normal, 1=IMU Timeout
wifi_error	bool	0=Normal, 1=Heartbeat Timeout
driver_heat_warn	bool	0=Normal, 1=Driver Overtemperature
driver_error	bool	0=Normal, 1=Driver Failure
motor_heat_warn	bool	0=Normal, 1=Motor Overtemperature
battery_low_warn	bool	0=Normal, 1=Low battery
battery_heat_warn	bool	0=Normal, 1= Battery overheating
gpio_error	bool	0=Normal, 1=GPIO error
cpu_heat_warn	bool	0=Normal, 1= CPU overheating
cpu_freq_warn	bool	0=normal, 1=CPU frequency too low

Error Type	Data Type	Value of the bit
reserved	bool[22]	Reserve

1.3.2 Robot Motion Status

Code	Type	Data content	Frequency
0x1009	1	Structure `MotionStateData` (See below)	200 Hz

```
struct MotionStateData{
  uint8_t basic_state;
  uint8_t gait_state;
  float max_forward_vel;
  float max_backward_vel;
  float leg_odom_pos[3];
  float leg_odom_vel[3];
  float robot_distance;
  unsigned touch_state;
  union {
    struct{
      uint32_t narrow_walk : 1;
      uint32_t pose_safe_flag : 1;
      uint32_t joint_limit_flag: 1;
      uint32_t state_reserved : 29;
    }control_state_bit;
    uint32_t control_state;
  };
  union{
    struct{
      uint8_t auto_charge_state;
      uint8_t pos_ctrl_state;
      uint8_t task_reserved[8];
    }task_state_list;
    uint8_t task_state[10];
  };
};
```

Fields	Type	Meaning
basic_state	uint8_t	Robot basic state[4]
gait_state	uint8_t	Current gait[5]

Fields	Type	Meaning
max_forward_vel	float	Deprecated, meaningless
max_backward_vel	float	Deprecated, meaningless
leg_odom_pos	float[3]	Robot position and orientation in world coordinate system[6]
leg_odom_vel	float[3]	Robot velocity in body coordinate system[7]
robot_distance	float	Mileage since last boot(cm)
touch_state	unsigned	Reserve
control_state	uint32_t	Reserve
task_state	uint8_t[10]	Invalid parameter, used only as placeholder[8]

[4]The value of field `basic_state` corresponds to the basic status of the robot as follows:

Value	Robot basic state
0	Sitting State
1	Sit-to-Stand State
2	Initial Standing State
3	Torque-controlled Standing State
4	Stepping State
5	Stand-to-Sit State
6	Emergency Stop/Fall State

[Note] Refer to 1.2.2 for the transformation relationship of the basic state of the robot.

[5]The value of field `gait_state` corresponds to the robot gait as follows:

Value	Gait
0	Walk
1	Off-Road
2	Slope
3	Run
6	Stair (elevation map mode = solid ground/grid ground/stair without rises)
7	Stair (elevation map mode= multi-frame mode)
8	45°Stair (elevation map mode= multi-frame mode)
32	L-Walk
33	Mountain
34	Silent

[6]The field `leg_odom_pos[3]` contains the following elements:

```
float leg_odom_pos[3] = {x,y,yaw};
```

Element	Meaning
x	Real-time X-coordinate of the robot in the world coordinate system(m)
y	Real-time Y-coordinate of the robot in the world coordinate system(m)
yaw	Real-time yaw angle of the robot in world coordinate system(rad)

[Note] The origin of the world coordinate system is the position of the robot at the time of startup, and its axis orientation is consistent with the orientation of the body coordinate system of the robot at the time of startup.

[7]The field `leg_odom_vel[3]` contains the following elements:

```
float leg_odom_vel[3] = {x_vel,y_vel,yaw_vel};
```

Element	Meaning
x_vel	Velocity of the robot on the X-axis of the body coordinate system (m/s)

Element	Meaning
y_vel	Velocity of the robot on the Y-axis of the body coordinate system (m/s)
yaw_vel	Angular velocity of yaw Angle in robot body coordinate system(rad/s)

[8] The parameters of `task_state` are defined as follows:

Parameter	Type	Value
auto_charge_state	uint8_t	Deprecated, meaningless
pos_ctrl_state	uint8_t	0 = Exit position control mode 1 = Initializing position control mode 2 = Executing position control 3 = position control execution completed 4 = Abort position control but remain in position control mode
task_reserved[8]	uint8_t[8]	Reserve

1.3.3 Motion Control Sensor Data

Code	Type	Data content	Frequency
0x100A	1	Structure `ControllerSensorData` (See below)	200 Hz

```
struct ControllerSensorData{
  ImuSensorData imu_data;
  LegJointData joint_pos;
  LegJointData joint_vel;
  LegJointData joint_tau;
};
```

Fields	Type	Meaning
imu_data	Structure ImuSensorData[9]	IMU data

Fields	Type	Meaning
joint_pos	Structure LegJointData[10]	Joint position (rad)
joint_vel	Structure LegJointData	Joint velocity (rad/s)
joint_tau	Structure LegJointData	Joint torque (N·m)

[9]The structure `ImuSensorData` are defined as follows:

```
struct ImuSensorData {
  int32_t timestamp;
  union {
    float buffer_float[9];
    uint8_t buffer_byte[3][12];
    struct {
      float roll, pitch, yaw; //angle (°)
      float omega_x, omega_y, omega_z; //angular velocity (rad/s)
      float acc_x, acc_y, acc_z; //Acceleration (m/s²)
    };
  };
};
```

[10]The structure `LegJointData` are defined as follows:

```
struct LegJointData{
  union{
    float data[12];
    struct{
      float fl_hipx_value, fl_hipy_value, fl_knee_value;
      float fr_hipx_value, fr_hipy_value, fr_knee_value;
      float hl_hipx_value, hl_hipy_value, hl_knee_value;
      float hr_hipx_value, hr_hipy_value, hr_knee_value;
    };
  };
};
```

Each leg of Jueying X30 consists of 3 joints: HipX (the hip joint for abduction and adduction), HipY (the hip joint for flexion and extension), and Knee.

The order of the 12 joints is FL_HipX, FL_HipY, FL_Knee, FR_HipX, FR_HipY, FR_Knee, HL_HipX, HL_HipY, HL_Knee, HR_HipX, HR_HipY, HR_Knee (F, H, L and R means front,

hind, left and right respectively).

1.3.4 Motion Control System Status

Code	Type	Data content	Frequency
0x100B	1	Structure `ControllerSafeData` (See below)	1 Hz

```
struct ControllerSafeData{
  float motor_temperature[12];
  uint8_t driver_temperture[12];
  CpuInfo cpu_info_;
};
```

Fields	Type	Meaning
motor_temperature	float[12]	Motor temperature of 12 joints (°C)
driver_temperture	uint8_t[12]	Driver temperature of 12 joints (°C)
cpu_info_	Structure CpuInfo[11]	CPU status

[11]The structure `CpuInfo` are defined as follows:

```
struct CpuInfo{
  float temperature;
  float frequency;
};
```

Fields	Type	Meaning
temperature	float	CPU temperature (°C)
frequency	float	CPU Clock Rate (MHz)

1.3.5 Battery Information

Code	Type	Data content	Frequency
0x21050F0A	1	Structure `BatterySensorData` (See below)	0.5 Hz

```
struct BatterySensorData {
  uint16_t voltage;
```

```
  int16_t  current;
  uint16_t remaining_capacity;
  uint16_t nominal_capacity;
  uint16_t cycles;
  uint16_t production_date;
  uint16_t balanced_low;
  uint16_t balanced_high;
  uint16_t protected_state;
  uint8_t software_version;
  uint8_t battery_level;
  uint8_t mos_state;
  uint8_t battery_quantity;
  uint8_t battery_ntc;
  float battery_temperature[4];
};
```

If Deeprcs Version is earlier than 1.10.19, the fields are defined as below:

Fields	Type	Meaning
voltage	uint16_t	Voltage (V)
current	int16_t	Current (A)
battery_level	uint8_t	Remain battery level (%)

If Deeprcs Version is 1.10.19 or later, the fields are defined as below:

Fields	Type	Meaning
voltage	uint16_t	Voltage (V)
current	int16_t	Current (10mA)
battery_level	uint8_t	Remain battery level (%)

1.3.6 Body Height Status

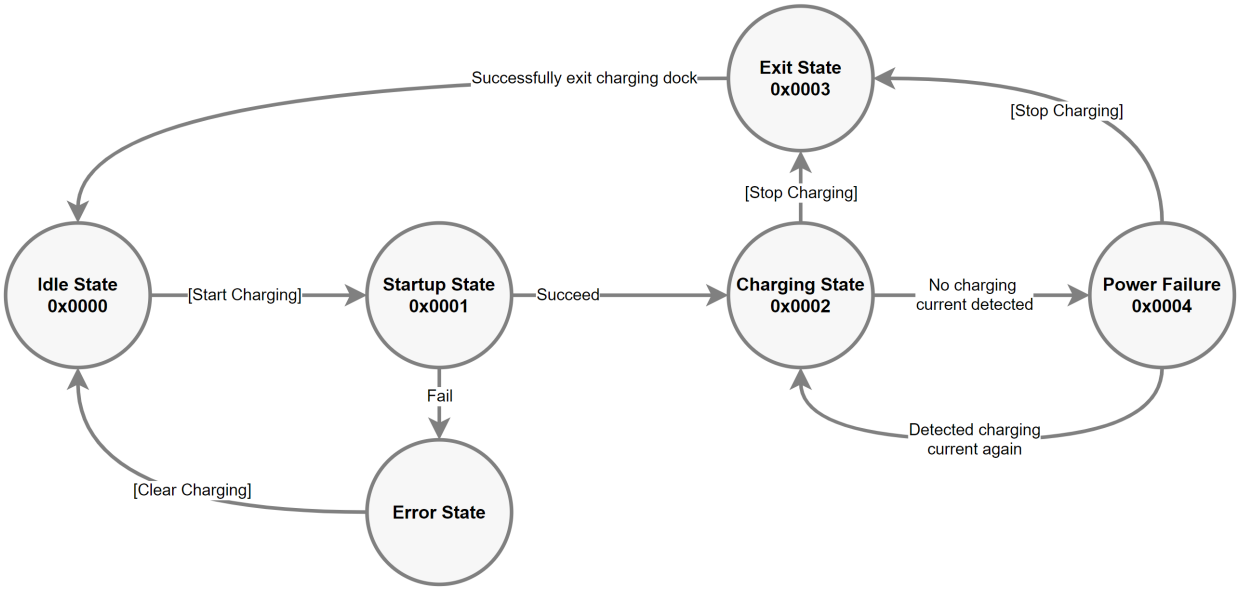
Code	Type	Value
0x11050F08	0	-1=Crawl，0=Normal

1.4 Command Set of Autonomous Charging

Autonomous Charging mainly includes the following states:

1. Idle State: The robot is waiting for the Start Charging Command, and is not executing a charging task.
2. Startup State: The robot enters the charging dock, adjusts its posture, and lies down on the charging dock after it receives the Start Charging Command.
3. Charging State: The robot is actively charging using the charging dock.
4. Exit State: The robot stands up and leaves the charging dock after it receives the Stop Charging Command.
5. Power Failure: If the robot fails to detect the charging current during charging, it will enter this state until the charging current is detected again.
6. Error State: The state in which the robot encounters anomalies during startup.

The transition relationships between each state are illustrated in the following diagram.



Autonomous charging command adopts a request-response interaction mechanism.

Users can send a request command to perception host `192.168.1.105:3333` via UDP and then the perception host will handle the request received and respond with the current state of autonomous charging.

The request commands related to autonomous charging are as follows:

Command	Code	Value	Type	Function
Start Charging	0x91910250	1	0	Make the robot enter the charging dock to charge
Stop Charging	0x91910250	0	0	Make the robot exit the charging dock
Clear Charging	0x91910250	2	0	Clear charging task and reset to idle state
Query State	0x91910253	0	0	Query the current state of auto-charging

After receiving the request with command code `0x91910250`, the perception host will respond with the following message:

Code	Value	Type
0x91910250	`charge_manager_state`	0

After receiving the request with command code `0x91910253`, the perception host will respond with the following message:

Code	Value	Type
0x91910253	`charge_manager_state`	0

The command value `charge_manager_state` corresponds to the autonomous charging state as follows:

Command Value	Autonomous Charging State
0x0000	Idle State
0x0001	Startup State
0x0002	Charging State

Command Value	Autonomous Charging State
0x0003	Exit State
0x0004	Power Failure
0x0005	Be charging with the charger
0x1002	Location Information Timeout
0x1003	Localization Startup Failure
0x1004	Failed to arrive above the charging dock
0x1005	No Location Information
0x1006	Location Information Spike
0x1007	No Current (sit down on the charging dock)
0x100a	No Current (exit the charging dock)

1.5 Command Set of Elevation Map Module

Elevation map module has the following two functions:

1. Footstep planning: While climbing stairs, the Elevation map module delivers 3D information about the surrounding obstacles to the motion program in order to plan the exact contact location and make stair climbing more stable.
2. Collision detection: In non-manual mode, the elevation map module detects the risk of collision in real time, and updates the input speed (axis commands from joystick or velocity commands issued by the navigation module) so that the robot slows down and stops or goes around the obstacles, thus avoiding collision.

1.5.1 Velocity Input Source

Developers can choose the velocity command input source of the elevation map: axis command of joystick or velocity command from navigation.

When the velocity command input source of elevation map is joystick, it is called assist mode, and the elevation map module will process the axis command issued by the joystick according to the collision detection situation, so as to realize the obstacle avoidance in the process of manual remote control.

When the velocity command input source of the elevation map is navigation module, it is called the navigation mode, and the elevation map module will process the velocity command issued by the navigation module to realize the obstacle avoidance in the navigation process.

1.5.1.1 Send Command to Switch Velocity Input Source

When using the collision detection function of the elevation map, the control mode should be switched to the non-manual mode first (refer to 1.2.4), and then the expected velocity input source can be set by sending the following UDP command to the perception host 192.168.1.105:43899:

Command	Code	Value	Type
Switching velocity input source	0x3101EE03	1= joystick，2= navigation module	0

[Note] Switching between manual and non-manual modes does not reset the elevation map's velocity input source when it occurs.

[Note] If the joystick app version is lower than v2.3.1, the joystick will automatically reset the elevation map's velocity input source to joystick when the control mode is switched back to manual mode from navigation mode or assist mode on the joystick.

1.5.1.2 Receive Messages of Current Velocity Input Source

The port 43899 of perception host will feed back the current velocity input source information at a frequency of 10 Hz after receiving any message from the client:

Command	Code	Value
0x3100EE03	0	1= joystick，2= navigation module

1.5.2 Elevation Map Mode

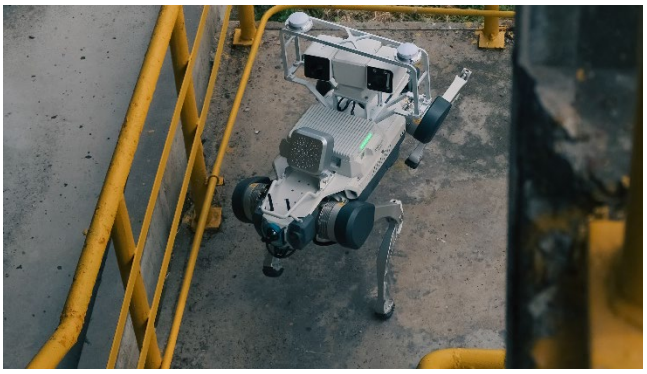
In the process of using the elevation map function, users need to set the correct elevation map mode for the elevation map according to the actual environment:

1. Before climbing stairs, to ensure that the footstep planning function works properly, you need to select the correct elevation map mode according to the type of stairs;
2. In assist mode or navigation mode, to make the elevation map collision detection function work properly, you need to select the correct elevation map mode according to the environment.

There are four types of elevation map modes: Solid Ground Mode, Grid Ground Mode, Stair Without Risers Mode and Multi-Frame Mode.

1.5.2.1 Command to Switch to Solid Ground Mode

The solid ground mode is suitable for scenarios where the robot's landing plane is solid, such as a normal solid concrete flat floor or concrete stairs.





Users can switch to solid ground mode by sending the following UDP message to the perception host:

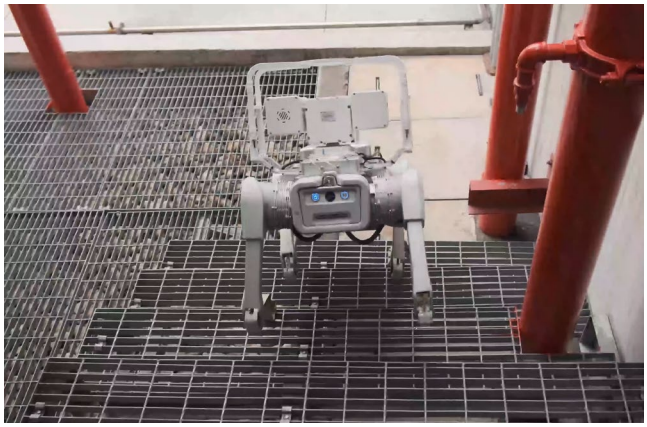
Code	Value	Type
0x3101EE01	3	0

[Note] If you need to switch from multi-frame mode to solid ground mode, you need to refer to 1.5.2.5 to execute the process of exiting from multi-frame mode.

1.5.2.2 Command to Switch to Grid Ground Mode

The grid ground mode is suitable for scenarios where the robot's landing plane is a grid-like pattern with gaps in between, such as an industrial platform or staircase made of steel grating panels.





Users can switch to grid ground mode by sending the following UDP message to the perception host:

Code	Value	Type
0x3101EE01	4	0

[Note] If you need to switch from multi-frame mode to grid ground mode, you need to refer to 1.5.2.5 to execute the process of exiting from multi-frame mode.

1.5.2.3 Command to Switch to Stair Without Risers Mode

The stair without risers mode is a special condition of solid ground, which refers to a staircase where the robot places its feet on a solid surface without risers.



Users can switch to stair without risers mode by sending the following UDP message to the perception host:

Code	Value	Type
0x3101EE01	5	0

[Note] If you need to switch from multi-frame mode to stair without risers mode, you need to refer to 1.5.2.5 to execute the process of exiting from multi-frame mode.

1.5.2.4 Command to Switch to Multi-Frame Mode

The muti-frame mode is suitable for scenarios including multiple types of ground surfaces. For example, if the robot moves from solid ground to grid ground, or from grid ground to solid ground, it should be marked as composite ground. The obstacle avoidance function based on the local elevation map is not supported on composite ground, and only composite ground stair gait and 45° stair gait are applicable on composite ground.

[Note] Multi-frame mode (including related gaits) is only supported by vmap (≥1.5.3) and needs to be used with jy_exe (≥1.10.28), launcher (≥0.0.5), fast-lio (≥1.0.3), and jycog-system (≥0.5.5).

The steps to switch to the multi-frame mode are as follows (the example code can be found in Appendix C):

1. Before switching to the multi-frame mode, you need to keep the robot standing still until it successfully switches to the multi-frame mode.
2. Send the following UDP message to 192.168.1.105:60000 to start the LiDAR-Inertial Odometry:

Code	Value	Type
0xBAA0001	1	0

The perception host will feed back the result of the execution once after receiving the above message:

Code	Value	Type
0xBAA0001	0=Succeed，-1=Failed	0

If it fails, it is necessary to exit from multi-frame mode (refer to 1.5.2.5) before attempting to start the LiDAR-Inertial Odometry again.

3. After you have successfully start the LiDAR-Inertial Odometry, you can send the following UDP message to 192.168.1.105:43899 to switch to multi-frame mode:

Code	Value	Type
0x3101EE01	20	0

4. After receiving the message to switch to multi-frame mode, the elevation map module switches its mode to the multi-frame preparation state, checks whether the LiDAR-Inertial Odometry is switched on successfully, and switches to multi-frame

mode after the LiDAR-Inertial Odometry is switched on. At this time, it can judge whether the elevation map mode has successfully switched to the multi-frame mode by receiving the UDP message described in 1.5.2.6. If the elevation map module continues to provide feedback that it is in the multi-frame preparation state for more than a certain period (the recommended timeout value is 10s), it can be regarded as a failure, and it is necessary to exit from the multi-frame mode (refer to 1.5.2.5) before attempting to switch to the multi-frame mode again from step 1.

5. Ensure that the elevation map mode is multi-frame mode before controlling the robot to start stepping and continue the movement.

1.5.2.5 Command to Switch to Other Modes From Multi-Frame Mode

While switching from multi-frame mode to other elevation map modes, the following UDP message needs to be sent to the perception host 192.168.1.105:60000 to turn off the LiDAR-Inertial Odometry after ensuring that it has been switched to other elevation modes:

Code	Value	Type
0xBAA0001	0	0

The perception host will feed back the execution result once after receiving the above message:

Code	Value	Type
0xBAA0001	0=Succeed，-1=Failed	0

1.5.2.6 Receive Messages of Current Elevation Map Mode

The port 43899 of perception host will feed back the current elevation map mode at a frequency of 10 Hz after receiving any message from the client:

Code	Type	Value
0x3100EE01	0	3= solid ground mode 4= grid ground mode 5= stair without risers mode 18= multi-frame preparation state 20= multi-frame mode

1.5.3 Obstacle Avoidance Modes

When the elevation map detects the risk of collision, it can be used to avoid the collision by decelerating to zero or detouring the obstacles.

The user can send the following command to 192.168.1.105:43899 to change obstacle avoidance mode:

Code	Value	Type
0x3101EE02	1=Decelerate, 2=Detour	0

The port 43899 of perception host will feed back the current obstacle avoidance mode of the elevation map at a frequency of 10 Hz after receiving any message from the client:

Code	Type	Value
0x3100EE02	0	1=Decelerate, 2=Detour

1.5.4 Obstacle Height Threshold

In the process of using the elevation map collision detection function, the elevation map module will regard objects on the ground higher than the obstacle height threshold as obstacles. Users can set the obstacle height threshold themselves by sending the following command to 192.168.1.105:43899:

Code	Type	Value
0x3100EE04	0	1= Set the height threshold to 8cm 2= Set the height threshold to 28cm

It should be noted that the user should adjust the obstacle height thresholds according to the gait used, and the recommended thresholds are shown in the table below:

Gait	Recommended obstacle height thresholds
Walk/Slope	8cm
Off-road/Stair/45°Stair	28cm

2 Perception Host ROS Topics

This section describes ROS topics in the perception host (192.168.1.105).

2.1 Topics of Sensor Driver

The sensor data are all published to the topics upon robot power-up, and developers can subscribe to topics for sensor data.

Topic Name	Description	Message Type	Frequency
/imu/data	Data of built-in IMU	sensor_msgs/Imu	200Hz
/livox/lidar	Point cloud data from LiDAR in a customized format	livox_ros_driver2/CustomMsg	10Hz
/lidar_points	Point cloud data of common format	sensor_msgs/PointCloud2	10Hz

The message type `livox_ros_driver2/CustomMsg` of topic `/livox/lidar` is a custom message type:

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
uint64 timebase
uint32 point_num
uint8 lidar_id
uint8[3] rsvd
livox_ros_driver2/CustomPoint[] points
  uint32 offset_time
  float32 x
  float32 y
  float32 z
  uint8 reflectivity
  uint8 tag
  uint8 line
```

Before using the `rostopic echo` command to view this topic message, you need to execute the following command to get the definition of the topic message type:

```
source ~/jy_cog/transfer/setup.bash
```

Developers who want to subscribe to the topic in their own programs need to define the structure of this custom message type in codes.

2.2 Topics of Basic Status Information

The basic status information is published to the topics in this section upon robot power-up, and developers can subscribe to these topics.

2.2.1 Battery Information

Topic Name	Description	Message Type
/battery/level	robot's current remaining power (%)	std_msgs/UInt8
/battery/current	Robot current (A or 10mA)	std_msgs/Float32
/battery/voltage	Robot voltage (V)	std_msgs/Float32
/battery_state	Robot Battery Information	message_transformer_cpp/BatteryData

The messaged type `message_transformer_cpp/BatteryData` of topic `/battery_state` is a custom message type:

```
uint16 voltage
int16 current
uint16 remaining_capacity
uint16 nominal_capacity
uint16 cycles
uint16 production_date
uint16 balanced_low
uint16 balanced_high
uint16 protected_state
uint8 software_version
uint8 battery_level
uint8 mos_state
uint8 battery_quantity
uint8 battery_ntc
float32[] battery_temperature
```

Some of the fields are specifically defined below:

Field Name	Meaning

voltage	Robot voltage（V）
current	Robot current (A or 10mA)
remaining capacity	Robot remaining capacity（10mAh）
nominal capacity	Robot nominal capacity（10mAh）
battery level	Percentage of remaining capacity（%）

You need to execute the following command to get the definition of the topic message type before using the `rostopic echo` command to view this topic message:

```
source ~/jy_cog/transfer/setup.bash
```

Developers who wish to subscribe to these topics in their own programs need to define the structure of this custom message type in codes.

[Note] If the version of jy_exe is lower than v1.10.19, the unit of current in the topics introduced in this section is A. If the version of jy_exe is v1.10.19 or higher, the unit of current in the topics introduced in this section is 10mA.

2.2.2 Motion State

2.2.2.1 Control Mode

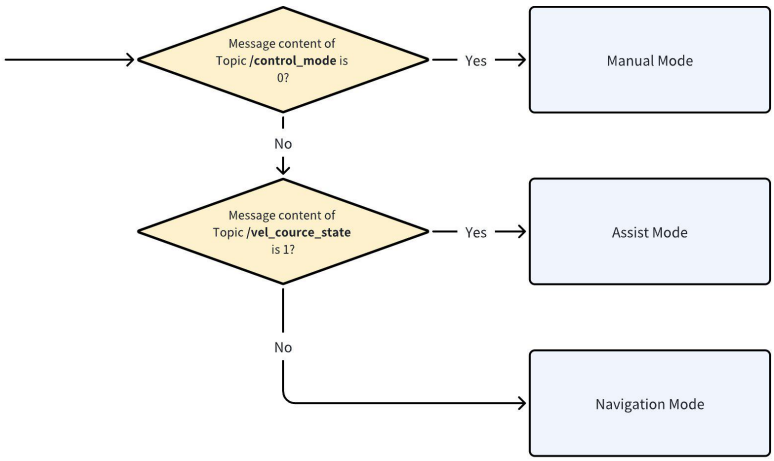
Topic Name	Description	Message Type
/control mode	control mode	std_msgs/UInt8

The control modes corresponding to the message contents of topic `/control_mode` are shown in the following table:

Topic Message	Corresponding control mode
0	Manual mode
1	Non-manual mode

The motion program determines the source of the velocity command to be executed depending on the control mode it is currently in: when in manual mode, the motion program directly executes the original velocity command from the joystick; when in

non-manual mode, the motion program executes the joystick velocity command or the navigation velocity command corrected by elevation map module. Depending on the source of the elevation map velocity input, the non-manual mode is differentiated into assist mode and navigation mode, which can be distinguished by the user through the topic `/vel_source_state` (refer to 2.4.3).



2.2.2.2 Basic Motion State

Topic Name	Description	Message Type
/robot_basic_state	Robot basic motion state	std_msgs/Int32

The **basic motion state** corresponding to message contents of this topic are shown in the following table:

Topic message	Corresponding basic motion state
0	Sitting State
1	Sit-to-Stand State
2	Initial Standing State
3	Torque-controlled Standing State
4	Stepping State
5	Stand-to-Sit State
6	Emergency Stop/Fall State

2.2.2.3 Gait

Topic Name	Description	Message Type
/robot_gait_state	Current robot gait	std_msgs/Int32

The **gait** corresponding to the message contents of topic `/robot_gait_state` are shown in the following table:

Topic Message	Corresponding gait
0	Walk
1	Off-Road
2	Slope
3	Run
6	Stair (elevation map mode = solid ground/grid ground/stair without rises)
7	Stair (elevation map mode= multi-frame mode)
8	45°Stair (elevation map mode= multi-frame mode)
32	L-Walk
33	Mountain
34	Silent

2.2.2.4 Velocity

Topic Name	Description	Message Type
/robot_velocity	Robot current velocity	geometry_msgs/Twist

2.2.2.5 Movement Information

Topic Name	Description	Message Type
/leg_odom	Robot coordinate information in the world coordinate system	nav_msgs/Odometry

2.2.2.6 Motion Time

Topic Name	Description	Message Type
/run_time/current_motion_time	Cumulative walking time after the current power-up (s)	std_msgs/Int64
/run_time/total_motion_time	Total walking time (s)	std_msgs/Int64

[Note] If the version of the running program jy_exe is 1.10.28 or earlier, the running time will not be updated until after the next rise.

2.2.3 Joint State

2.2.3.1 Joint Temperature

Topic Name	Description	Message Type
/driver_temperature	Driver temperature for 12 joints (°C)	std_msgs/UInt8MultiArray
/motor_temperature	Motor temperature for 12 joints (°C)	std_msgs/Float32MultiArray

Each leg of Jueying X30 consists of 3 joints: HipX (the hip joint for abduction and adduction), HipY (the hip joint for flexion and extension), and Knee.

The order of the 12 joints is generally FL_HipX, FL_HipY, FL_Knee, FR_HipX, FR_HipY, FR_Knee, HL_HipX, HL_HipY, HL_Knee, HR_HipX, HR_HipY, HR_Knee (F, H, L and R means front, hind, left and right respectively).

2.2.3.2 Joint Data

Topic Name	Description	Message Type
/joint_states	Joint State	sensor_msgs/JointState

2.2.4 Running State

2.2.4.1 Mileage and Running Time

Topic Name	Description	Message Type
/mileage/current_mileage	Mileage after the current start-up (cm)	std_msgs/Int32
/mileage/total_mileage	Total mileage (cm)	std_msgs/Int32
/run_time/current_run_time	Running time after the current power-up (s)	std_msgs/Int64
/run_time/total_run_time	Total run time (s)	std_msgs/Int64

2.2.4.2 CPU State

Topic Name	Description	Message Type

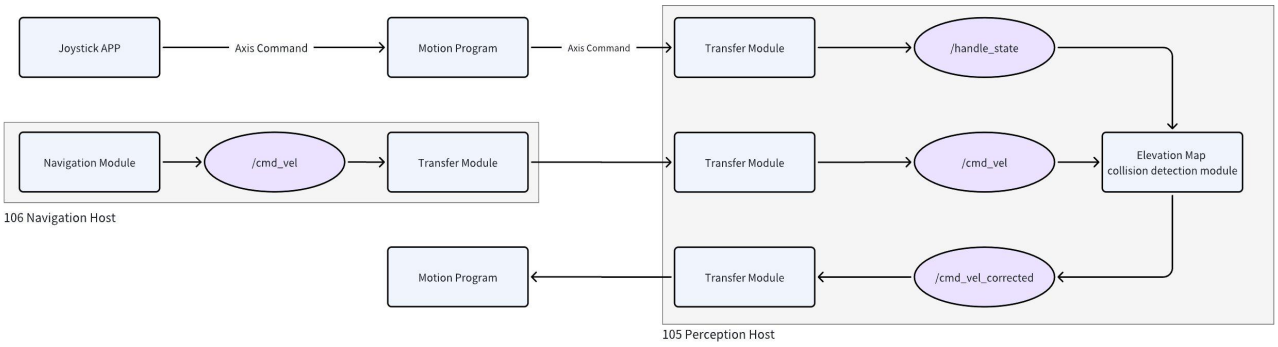
/motion_cpu/frequency	Motion Host CPU main frequency (MHz)	std_msgs/Float32
/motion_cpu/temperature	Motion Host CPU temperature (°C)	std_msgs/Float32

2.3 Velocity-Related Topics

There are three velocity-related topics included in Perception Host:

1. the velocity command `/handle_state` issued by the joystick;
2. the velocity command `/cmd_vel` issued by the navigation module;
3. the velocity command `/cmd_vel_corrected` corrected by the elevation map collision detection module.

In assist or navigation mode, the motion program executes the velocity command published to `/cmd_vel_corrected` (shown below).



2.3.1 Velocity Commands from Joystick

The velocity commands from joystick will be published to the topic `/handle_state` when the joystick is triggered.

Topic Name	Description	Message Type
/handle_state	Joystick velocity commands	geometry_msgs/Twist

2.3.2 Velocity Commands from Navigation Module

When the navigation module performs path planning, the velocity command output by the navigation module will be published to the topic `/cmd_vel`.

Topic Name	Description	Message Type
/cmd_vel	Velocity planned by the navigation module	geometry_msgs/Twist

2.3.3 Velocity Commands Corrected by Elevation Map Module

When a raw velocity command from the joystick or navigation module is published, the elevation map will correct it according to the real-time collision risk and publish to the topic `/cmd_vel_corrected`. When there is no raw velocity command input from the joystick or navigation module, the developer can also publish messages directly to this topic, as appropriate, to make the motion program execute a velocity command that has not been processed by the elevation map module.

Topic Name	Description	Message Type
/cmd_vel_corrected	Velocity commands corrected by elevation map module	geometry_msgs/Twist

2.4 Topics of Elevation Map Module

In non-manual mode, the elevation map collision detection module corrects the input velocity according to the real-time local terrain conditions, and the motion program executes the corrected velocity, thus avoiding collisions.

2.4.1 Access to Elevation Map Data

The elevation map data is published to this topic upon robot power-up, and developers can subscribe to the topic message to get the data.

Topic Name	Description	Message Type
/deeprobotics_local_height_map	Obstacle point cloud in robot center-of-mass coordinate system	sensor_msgs/PointCloud2

_mid360/obstacle_pointcloud		

2.4.2 Obstacle Avoidance Mode

When a collision risk is detected by the elevation map, deceleration or detour strategy can be used to avoid the collision.

2.4.2.1 Choose Obstacle Avoidance Mode

The developer can use this topic to choose the strategy adopted by the elevation map.

Topic Name	Description	Message Type
/brake_mode	Choose obstacle avoidance mode	std_msgs/Int32

If developers want to change the obstacle avoidance mode, corresponding topic messages should be published to the topic, as shown in the table below:

Mode to be set	Topic messages to be published
Decelerate	1
Detour	2

2.4.2.2 Check Obstacle Avoidance Mode

The obstacle avoidance mode information is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current mode.

Topic Name	Description	Message Type
/brake_mode_state	Check current obstacle avoidance mode	std_msgs/Int32

The message content corresponds to obstacle avoidance mode as shown in the table below:

Obtained topic message	Corresponding mode
1	Decelerate
2	Detour

2.4.3 Velocity Input Source

The developer can change the velocity command input source of the elevation map: joystick or navigation module.

When the velocity command input source of elevation map is set to joystick, it is called assist mode, and the elevation map module will process the joystick command issued by the joystick according to the collision detection situation, so as to realize the obstacle avoidance in the process of manual remote control.

When the velocity command input source of the elevation map is set to the navigation module, it is called the navigation mode, and the elevation map module will process the velocity command issued by the navigation module to realize the obstacle avoidance in the navigation process.

2.4.3.1 Choose Velocity Input Source

When using the collision detection function of elevation map, the control mode should be switched to non-manual mode first, and then a message can be published to the topic `/vel_source` to choose the required velocity input source.

Topic Name	Description	Message Type
/vel_source	Choose velocity input source	std_msgs/Int32

The velocity input sources represented by messages published to the topic are shown as the table below:

Velocity input source to be set	Topic messages to be posted
Joystick	1
Navigation module	2

[Note] Switching between manual and non-manual modes does not reset the elevation map's velocity input source when it occurs.

[Note] If the joystick app version is lower than 2.3.1, the joystick will automatically reset the

elevation map's velocity input source to the joystick when switching back to manual mode from navigation mode or assist mode on the joystick.

2.4.3.2 Check Current Velocity Input Source

The velocity input source information is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current source.

Topic Name	Description	Message Type
/vel_source_state	Check current velocity input source	std_msgs/Int32

The velocity input sources corresponding to the message contents are shown in the table below:

Obtained topic message	Corresponding velocity input source
1	Joystick
2	Navigation module

2.4.4 Elevation Map Mode

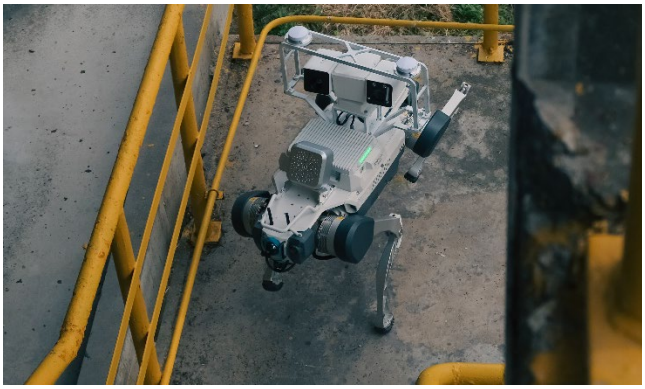
In the process of using the elevation map function, users need to set the correct elevation map mode for the elevation map according to the actual environment:

1. Before climbing stairs, in order to ensure that the footstep planning function works properly, you need to select the correct elevation map mode according to the type of stairs;
2. In assist mode or navigation mode, in order to make the elevation map collision detection function work properly, you need to select the correct elevation map mode according to the environment.

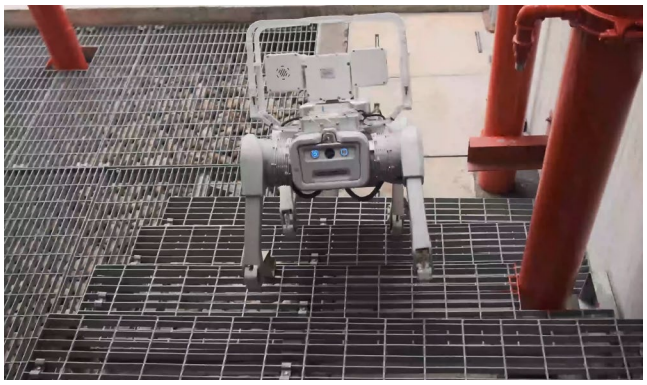
There are four types of elevation map modes: Solid Ground Mode, Grid Ground Mode, Stair Without Risers Mode and Multi-Frame Mode.

2.4.4.1 Switch to Solid Ground/Grid Ground/Stair Without Risers Mode

The solid ground mode is suitable for scenarios where the robot's landing plane is solid, such as a normal solid concrete flat floor or concrete stairs.




The grid ground mode is suitable for scenarios where the robot's landing plane is a grid-like pattern with gaps in between, such as an industrial platform or staircase made of steel grating panels.

The stair without risers mode is a special condition of solid ground, which refers to a staircase where the robot places its feet on a solid surface without risers.



This topic allows developers to choose the appropriate mode for elevation map before moving on to environments with different types of landing surfaces.

Topic Name	Description	Message Type
/height_map_mode	Choose elevation map mode	std_msgs/Int32

Setting to different elevation map modes corresponds to the topic messages that need to be published to the topic as shown in the table below:

Elevation map mode to be set	Topic messages to be published
Solid Ground Mode	3
Grid Ground Mode	4
Stair Without Risers Mode	5

2.4.4.2 Switch to Multi-Frame Mode

The muti-frame mode is suitable for scenarios including multiple types of ground surfaces. For example, if the robot moves from solid ground to grid ground, or from grid ground to solid ground, it should be marked as composite ground. The obstacle avoidance function based on the local elevation map is not supported on composite

ground, and only composite ground stair gait and 45° stair gait are applicable on composite ground.

[Note] Multi-frame mode (including related gaits) is only supported by vmap (≥1.5.3) and needs to be used with jy_exe (≥1.10.28), launcher (≥0.0.5), fast-lio (≥1.0.3), and jycog-system (≥0.5.5).

The steps to switch to the multi-frame mode are as follows (the example code can be found in Appendix C):

1. Before switching to the multi-frame mode, you need to keep the robot standing still until it successfully switches to the multi-frame mode.
2. Send the following UDP message to 192.168.1.105:60000 to start the LiDAR-Inertial Odometry:

Code	Value	Type
0xBAA0001	1	0

The perception host will feed back the result of the execution once after receiving the above message:

Code	Value	Type
0xBAA0001	0=Succeed，-1=Failed	0

If it fails, it is necessary to exit from multi-frame mode (refer to 2.4.4.3) before attempting to start the LiDAR-Inertial Odometry again.

3. After you have successfully start the LiDAR-Inertial Odometry, you can publish message to the topic `/height_map_mode` to switch to multi-frame mode:

Topic Name	Description	Message Type
/height_map_mode	Choose elevation map mode	std_msgs/Int32

The topic messages that need to be published to this topic are shown in the following table:

Elevation map mode to be set	Topic messages to be published
Multi-Frame Mode	20

4. After receiving the message to switch to multi-frame mode, the elevation map module switches its mode to the multi-frame preparation state, checks whether the LiDAR-Inertial Odometry is switched on successfully, and switches to multi-frame mode after the LiDAR-Inertial Odometry is switched on. Developers can subscribe to the topic in 2.4.4.4 to judge whether the elevation map mode has successfully switched to the multi-frame mod. If the elevation map module continues to provide feedback that it is in the multi-frame preparation state for more than a certain period (the recommended timeout value is 10s), it can be regarded as a failure, and it is necessary to exit from the multi-frame mode (refer to 2.4.4.3) before attempting to switch to the multi-frame mode again from step 1.
5. Ensure that the elevation map mode is multi-frame mode before controlling the robot to start stepping and continue the movement.

2.4.4.3 Switch From Multi-Frame Mode To Other Modes

While switching from multi-frame mode to other elevation map modes, the following UDP message (refer to 1.1) needs to be sent to the perception host 192.168.1.105:60000 to turn off the LiDAR-Inertial Odometry after ensuring that it has been switched to other elevation modes:

Code	Value	Type
0xBAA0001	0	0

The perception host will feed back the execution result once after receiving the above message:

Code	Value	Type
0xBAA0001	0=Succeed，-1=Failed	0

2.4.4.4 Check Elevation Map Mode

The elevation map mode information is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current mode.

Topic Name	Description	Message Type
/height_map_mode_state	Check elevation map mode	std_msgs/Int32

The elevation map modes corresponding to the message contents are shown in the following table:

Obtained topic message	Corresponding elevation map mode
3	Solid Ground Mode
4	Grid Ground Mode
5	Stair Without Risers Mode
18	Multi-Frame Preparation State
20	Multi-Frame Mode

2.4.5 Obstacle Height Threshold

In the process of using the elevation map collision detection function, the elevation map module will regard objects on the ground higher than the obstacle height threshold as obstacles.

2.4.5.1 Configure Obstacle Height Threshold

The developer should set different obstacle height thresholds according to the ability of different gaits to cross obstacles. In general, the obstacle height threshold is set to 0.25

for stair gait, 0.12 for slope gait, and 0.08 for walking gait. Since the grass will be regarded as an obstacle, the threshold can be increased appropriately according to the actual situation when crossing the grass.

Topic Name	Description	Message Type
/step_z_max	Configure the obstacle height threshold(m)	std_msgs/Float64

2.4.5.2 Check Obstacle Height Threshold

The obstacle height threshold is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current threshold.

Topic Name	Description	Message Type
/step_z_max_state	Check current obstacle height threshold	std_msgs/Float64

2.4.6 Velocity Correction Time Threshold

When the elevation map predicts that a collision will occur within the time threshold, it will correct the original velocity commands to avoid collisions.

2.4.6.1 Configure Velocity Correction Time Threshold

The developer can modify the time threshold via this topic.

Topic Name	Description	Message Type
/slow_t	Configure Velocity Correction Time Threshold(s)	std_msgs/Float64

2.4.6.2 Check Velocity Correction Time Threshold

The time threshold is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current threshold.

Topic Name	Description	Message Type
/slow_t_state	Check Velocity Correction Time Threshold (s)	std_msgs/Float64

2.4.7 Braking Time Threshold

When the elevation map predicts that a collision will occur within the time threshold, it will simply correct the raw velocity to zero.

2.4.7.1 Configure Braking Time Threshold

The developer can modify the time threshold via this topic.

Topic Name	Description	Message Type
/stop_t	Configure Braking Time Threshold(s)	std_msgs/Float64

2.4.7.2 Check Braking Time Threshold

The time threshold is published to this topic upon robot power-up, and developers can subscribe to the topic message to check the current threshold.

Topic Name	Description	Message Type
/stop_t_state	Check Braking Time Threshold(s)	std_msgs/Float64

2.5 Topics of Autonomous Charging

The autonomous charging state information are published to the topic in this section upon robot power-up, and developers can subscribe to this topic.

Topic Name	Description	Message Type
/charge_manager_state	Check autonomous charging state	std_msgs/Int32

The message contents of the topic `/charge_manager_state` correspond to the autonomous charging states as shown in the table below:

Obtained topic messages	Corresponding autonomous charging state
0	Idle State
1	Startup State
2	Charging State
3	Exit State
4	Power Failure

4098	Location Information Timeout
4099	Localization Startup Failure
4100	Failed to arrive above the charging dock
4101	No Location Information
4102	Location Information Spike
4103	No Current (sit down on the charging dock)
4106	No Current (exit the charging dock)

3 Navigation Host ROS Topics

This section describes ROS topics in navigation host (192.168.1.106).

3.1 Topics of Sensor Driver

The sensor driver topics in navigation host is the same as the perception host, please refer to section 2.1.

3.2 Topics of Basic Status Information

The basic status information topics in navigation host is the same as the perception hosts, please refer to section 2.2.

3.3 Topics of Localization

3.3.1 Reset Robot Pose

This topic is activated when the localization program is started, and developers can publish messages to the topic to reset the robot pose.

Topic Name	Description	Message Type
/initialpose	Reset robot pose	geometry_msgs/PoseWithCovarianceStamped

3.3.2 Change Global Map

The topic is activated when the localization program is started, and developers can publish messages to the topic to change the global point cloud map for the localization.

Topic Name	Description	Message Type
/map_request/pcd	Specify the map(.pcd) path to be used by the localization	std_msgs/String

3.3.3 Obtain Real-time Pose

The real-time robot pose message is published to the topic below as soon as the localization program is launched, and developers can subscribe to the topic to get the real-time pose of the robot.

Topic Name	Description	Message Type
/odom	Real-time robot pose information	nav_msgs/Odometry

3.3.4 Get Global Point Cloud Map Data

The global point cloud map data is published to the topic below as soon as the localization program is launched, and developers can subscribe to the topic to get the map data.

Topic Name	Description	Message Type
/globalmap	Global point cloud map data	sensor_msgs/PointCloud2

3.3.5 Obtain Location Status

The topic message is published as soon as the localization program is launched, and developers can subscribe to the topic to check if the location is available:

Topic Name	Description	Message Type
/location_status	Location Status	std_msgs/Int8

The topic `/location_status` message content corresponds to the location status as shown in the following table:

Obtained topic messages	Corresponding location status
0	Localization is normal
1	Location is not available

3.4 Topics of Navigation

Navigation module has two modes: Auto Navigation and Straight Navigation. When using the Straight navigation, the path will be planned globally in a straight line. When using the Auto navigation, the navigation module will plan a path in the passable area and automatically bypass obstacles.

3.4.1 Get Occupancy Grid Map Data

The topic message is published as soon as the navigation program is launched and developers can subscribe to the topic to get the occupancy grid map data used in navigation:

Topic Name	Description	Message Type
/map	Occupancy Grid Map Data	nav_msgs/OccupancyGrid

3.4.2 Specify Auto Navigation Goals

This topic is activated when the navigation program is launched, and developers can publish messages to the topic to specify auto navigation goals:

Topic Name	Description	Message Type
/move_base/goal	Specify an auto navigation goal	MoveBaseActionGoal

Where the message type `MoveBaseActionGoal` is a custom message type:

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
actionlib_msgs/GoalID goal_id
  time stamp
  string id
move_base_msgs/MoveBaseGoal goal
  geometry_msgs/PoseStamped target_pose
    std_msgs/Header header
```

```
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
```

Before using the `rostopic echo` command to check this topic message, you need to execute the following command to get the definition of the topic message type:

```
source ~/jy_cog/broker/setup.bash
```

If the above command does not work, execute the following command to try again to get the definition of the topic message type:

```
source ~/jy_cog/nav/setup.bash
```

Developers who need to subscribe to the topic in their own programs need to define the structure of this custom message in codes.

3.4.3 Specify Straight Navigation Goals

This topic is activated when the navigation program is launched, and developers can publish messages to the topic to specify straight navigation goals:

Topic Name	Description	Message Type
/move_base_direct_line/goal	Specify a straight navigation goal	MoveBaseDirectLineActionGoal

Where the message type `MoveBaseDirectLineActionGoal` is a custom message type:

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
actionlib_msgs/GoalID goal_id
  time stamp
```

```
  string id
move_base_msgs/MoveBaseDirectLineGoal goal
  geometry_msgs/PoseStamped target_pose
    std_msgs/Header header
      uint32 seq
      time stamp
      string frame_id
    geometry_msgs/Pose pose
      geometry_msgs/Point position
        float64 x
        float64 y
        float64 z
      geometry_msgs/Quaternion orientation
        float64 x
        float64 y
        float64 z
        float64 w
  bool is_backward_mode
```

Before using the `rostopic echo` command to check this topic message, you need to execute the following command to get the definition of the topic message type:

```
source ~/jy_cog/broker/setup.bash
```

If the above command does not work, execute the following command to try again to get the definition of the topic message type:

```
source ~/jy_cog/nav/setup.bash
```

Developers who need to subscribe to the topic in their own programs need to define the structure of this custom message in codes.

3.4.4 Get Global Path Planned by Auto Navigation

After publishing an auto navigation goal, developer can subscribe to the topic below to get the global path planned by the auto navigation:

Topic Name	Description	Message Type
/move_base/GlobalPlanner/plan	Global path planned by auto navigation	nav_msgs/Path

3.4.5 Get Local Path Planned by Auto Navigation

After publishing an auto navigation goal, developer can subscribe to the topic below to get the local path planned by the auto navigation:

Topic Name	Description	Message Type
/move_base/LocalPlanner/local_plan	Local path planned by auto navigation	nav_msgs/Path

3.4.6 Get Global Path Intercepted By Auto Navigation Local Planner

After publishing an auto navigation goal, developer can subscribe to the topic below to get the global path intercepted by auto navigation local planner:

Topic Name	Description	Message Type
/move_base/LocalPlanner/global_plan	Global paths intercepted by auto navigation local planner	nav_msgs/Path

3.4.7 Get Obstacle Avoidance Status of Navigation

When the robot is navigating, the developer can subscribe to the topic to get the obstacle avoidance status of navigation:

Topic Name	Description	Message Type
/move_base/obs_state	Navigation Obstacle Avoidance Status	std_msgs/Int32

The message contents of topic `/move_base/obs_state` correspond to the obstacle avoidance status of navigation as shown in the table below:

Obtained topic messages	Corresponding obstacle avoidance status of navigation
0	No collision risks detected
1	Robot is avoiding obstacles

3.4.8 Obtain or Publish Navigation Velocity Commands

After publishing a navigation goal, developer can subscribe to the topic below to get the velocity commands published by navigation module:

Topic Name	Description	Message Type
/cmd_vel	Velocity command from navigation	geometry_msgs/Twist

The developer can also publish velocity commands planned by their own codes to this topic (see Appendix B for details of the maximum velocity for each gait), and in navigation mode, the topic message is forwarded to the elevation map module in the perception host for velocity correction, as described in 2.3.

Appendix A: UDP Message Parsing

Take the angle data of 12 joints as an example:

```
/// Original data, 108 bytes
/* 0209 0000 6000 0000 0100 0000 2efd 3ccf
 * 47e4 e1bf f130 4992 f49e e7bf c3d7 eefe
 * 8567 f13f 9ae4 b66d 6583 f1bf 24d8 f33c
 * 3d00 f13f 8c16 222a 0b1f 0140 e8ba aaaa
 * 3803 ef3f d009 0000 60bd d2bf db04 3353
 * 1150 e73f 7d0b 0000 72f0 e53f 0a3d 0cc3
 * f071 e73f c3d7 eefe 7b52 f93f
*/
/*****************************************************************************/
/// Content Parsing
//0209 0000      //command code = 0x0902
//6000 0000      //length of data = 96 bytes
//0100 0000      //zzzz=1 means complex command
//bbbbbbbbbb...................
/* The following is the data of the command which has 96 bytes and is an array with 12
elements of double type.
 * 2efd 3ccf 47e4 e1bf f130 4992 f49e e7bf
 * c3d7 eefe 8567 f13f 9ae4 b66d 6583 f1bf
 * 24d8 f33c 3d00 f13f 8c16 222a 0b1f 0140
 * e8ba aaaa 3803 ef3f d009 0000 60bd d2bf
 * db04 3353 1150 e73f 7d0b 0000 72f0 e53f
 * 0a3d 0cc3 f071 e73f c3d7 eefe 7b52 f93f
*/
//Robot joint angles
double joint_angle[12] = {-0.5591162726995316, -0.7381537301203293, 1.0877742727584383,
                          -1.0945791516990426, 1.0625584011991203, 2.1401580135014395,
                          0.9691432317102597, -0.2928085327149832, 0.7285238862024487,
                          0.6856012344363617, 0.7326587495353192, 1.5826377828902742};
```

Appendix B: Maximum Velocity of Each Gait

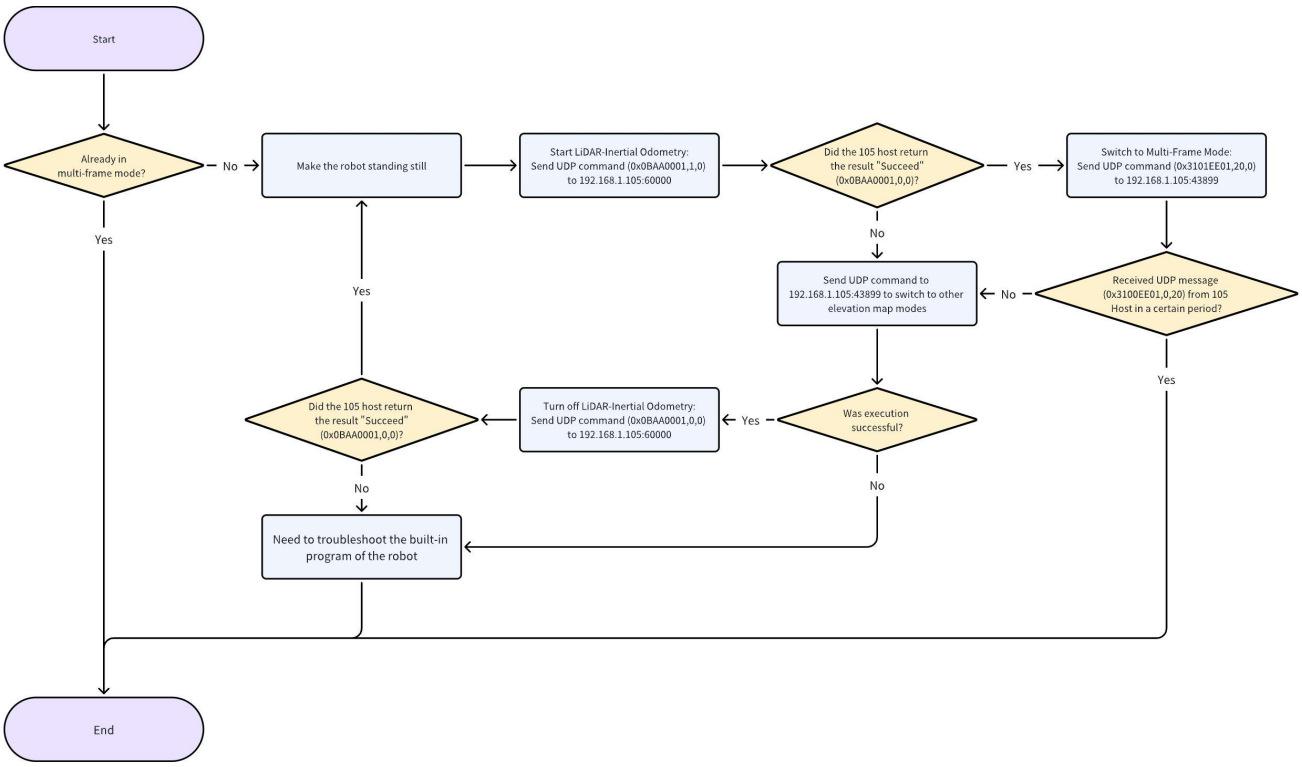
The maximum velocity limits for each gait when the robot is in a normal body height state are shown in the table below:

Gait (at normal height)	Maximum forward velocity	Maximum backward velocity	Maximum lateral velocity	Maximum angular velocity
Walk	1.2 m/s	0.75 m/s	0.15 m/s	0.45 rad/s
Slope	0.7 m/s	0.75 m/s	0.25 m/s	0.5 rad/s
Off-Road	0.3 m/s	0.4 m/s	0.1 m/s	0.5 rad/s
Stair (solid ground mode)	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
Stair (grid ground mode)	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
Stair (stair without rises mode)	0.3 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
Stair (multi-frame mode)	0.6 m/s	0.3 m/s	0.2 m/s	0.8 rad/s
45°Stair (multi-frame mode)	0.3 m/s	0.2 m/s	0.2 m/s	0.8 rad/s
L-Walk	1.0 m/s	1.0 m/s	0.5 m/s	1.2 rad/s
Mountain	1.0 m/s	1.0 m/s	0.8 m/s	1.2 rad/s

When the robot body is at crawl height, the maximum forward velocity of the walk gait and slope gait is reduced to 50% of the original.

Appendix C: How to Use Multi-Frame Mode via UDP

The basic procedure for switching to multi-frame mode as described in 1.5.2.4 is as follows:



1. The sample code to turn the LiDAR-Inertial Odometry on and off is shown below:

```
// Protocol Format: UDP (Request-Response Model: send a request and get a response back)
// Default IP: 192.168.1.105    PORT: 60000

/* When turning on the LiDAR-Inertial Odometry, the robot needs to stand still to ensure a
stable posture */

struct SimpleCommander {
  uint32_t command;
  uint32_t value;  // 0 = turn off; 1 = turn on
  uint32_t type;
};

int8_t Dispatch(int value) {
  SimpleCommander commander;
  commander.command = 0x0BAA0001;
```

```
  commander.value = value;
  commander.type = 0;

  auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
                    (struct sockaddr *)&addr_, sizeof(addr_));
  SimpleCommander recv_commander;
  if(ret > 0) {
    socklen_t addr_len = sizeof(addr_);
    ret = recvfrom(socket_fd_, &recv_commander, sizeof(SimpleCommander), 0,
                    (struct sockaddr *)&addr_, &addr_len);
    // recv_commander.value     0 = turn on/off succeed; -1 = turn on/off failed
    if(ret <= 0) return -1;
  } else return -1;
  return 0;
}
```

2. The sample code for checking the current elevation map mode is as follows:

```
// Protocol Format: UDP (Event-Driven Model: after receiving the first request, the robot
will keep replying elevation-map-related information)
// Default IP: 192.168.1.105    PORT: 43899

struct SimpleCommander {
  uint32_t command;
  uint32_t value;
  uint32_t type;
};

int8_t GetVMapData(int value) {
  SimpleCommander commander;
  commander.command = 0x3101EEFF;
  commander.value = 0;
  commander.type = 0;

  auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
                    (struct sockaddr *)&addr_, sizeof(addr_));
  SimpleCommander recv_commander;
  if(ret > 0) {
    socklen_t addr_len = sizeof(addr_);
    while(true) {
        ret = recvfrom(socket_fd_, &recv_commander, sizeof(SimpleCommander), 0,
                       (struct sockaddr *)&addr_, &addr_len);
        if(ret > 0) {
            if(recv_commander.command == 0x3100EE01) {
                // Current elevation map mode = recv_commander.value;
```

```
                // 3 = solid ground mode; 4 = grid ground mode; 5 = stair without risers
mode; 18 = multi-frame preparation state; 20 = multi-frame mode
            }
        }
    }
  } else return -1;
  return 0;
}
```

3. The sample code for changing the elevation map mode is as follows:

```
// Protocol Format: UDP (Event-Driven Model: after receiving the first request, the robot
will keep replying elevation-map-related information)
// Default IP: 192.168.1.105    PORT: 43899

/* Wait a moment to check if it switches to the corresponding elevation map mode after the
command is sent */
/* The current elevation map mode can be continuously obtained through the above Point 2 */
/* Point 2 and 3 use the same port */

struct SimpleCommander {
  uint32_t command;
  uint32_t value;  // 3 = solid ground mode; 4 = grid ground mode; 5 = stair without risers
mode; 18 = multi-frame preparation state ; 20 = multi-frame mode
  uint32_t type;
};

int8_t Dispatch(int value) {
  SimpleCommander commander;
  commander.command = 0x3101EE01;
  commander.value = value;
  commander.type = 0;

  auto ret = sendto(socket_fd_, (void*)&commander, 12, 0,
                    (struct sockaddr *)&addr_, sizeof(addr_));
  return 0;
}
```